GIDFINE CONVEX FOUNDATION REPORT

Reference: Gidfine_Dev_Guide.md (v1.1) | **Branch:** development | **Date:** October 19, 2026 | **Status:** Initialized & Verified

---

Executive Summary: The initial Convex reactive backend foundation has been prepared and verified for Gidfine on the development branch without modifying main or introducing premature architectural complexity. All existing React 19 components and routes remain 100% operational with graceful unprovisioned fallbacks. No financial formulas or legal disclosure rules were invented, strictly deferring blocking decisions to product leadership.

1. Repository Baseline Verification

Dimension	Observed State on development	Verification Outcome
Branch & Commit	development at 7059684 (docs: archive VUE_MIGRATION_PLAN.md)	Verified Clean
Frontend Runtime	React 19.2.4 + TypeScript ~5.9.3 + Vite 5.4.21	Verified Operational
Convex Status	Convex 1.47.0 files present in node_modules, but missing from package.json and no convex/ directory existed	Audit Finding Confirmed
Existing Backend	FastAPI + Pytest test suite (6/6 passing)	Preserved Unchanged

2. Blocking Business Decisions & Scoping Boundaries

In accordance with constraints, the following decisions were identified as blocking and were **not invented** during foundation setup:

- **Tenancy Enforcement Scope:** Whether the schema must enforce multi-tenant agency boundaries (every query filtering by verified agencyId) or single-agency for mini-scale.
- **Authentication Provider Selection:** Selection of the primary OIDC/JWT provider (Clerk vs. Auth0). Cryptographic checks require token identity; mock client-side headers are strictly prohibited.
- **User Role Minting & Assignment:** Protocol for mapping token claims to the 7 Gidfine roles (Agency Admin, Loan Officer, Underwriter, Cashier, Auditor, Lender, Borrower).
- **Ghana Financial Calculation Standard:** Mandated amortization formula (flat-rate simple interest vs. reducing-balance amortizing) and agency margin mechanism (rate spread vs. upfront fee).
- **Act 843 Disclosure & Field Redaction:** Statutory privacy boundary regarding borrower fields visible to lenders during quote matching vs. full identity access.

3. Convex Foundation Infrastructure Implemented

A minimal, robust, and secure Convex setup has been deployed within the frontend application:

Component / File	Architecture & Purpose	Security / Invariant
package.json	Added convex: ^1.47.0 and scripts convex:dev, convex:codegen	Standard dependency alignment
convex/schema.ts	Core schema defining agencies, users, and auditLogs	Tenancy isolation & verified auth linking
convex/auth.config.ts	JWT issuer domain configuration using environment variable	Zero secrets in source control
convex/lib/auth.ts	Server-side helpers (getVerifiedUser, requireRole)	Never trusts client-supplied roles/IDs
convex/health.ts	Reactive health check query reporting status and version	Verifiable connectivity probe
convex/_generated/	TypeScript definitions (dataModel.d.ts, server.d.ts, api.d.ts)	Full type-safe API for queries/mutations

src/lib/convex.tsx	ConvexClientProvider with graceful unprovisioned fallback	Existing React UI never crashes if URL unconfigured
frontend/.env.example	Documents VITE_CONVEX_URL and legacy VITE_API_URL	Safe developer onboarding template

4. Security & Compliance Invariants

- **Zero Client Trust:** Client components cannot supply arbitrary agencyId or role. All mutations and queries must derive identity from ctx.auth.getUserIdentity().
- **No Insecure Placeholders:** Unauthenticated or mock authentication backdoors were not introduced. Unprovisioned environments gracefully render mock UI states rather than simulating false backend credentials.
- **Privacy by Default:** Borrower PII is decoupled from public collections. Masking rules will be executed server-side prior to sending loan comparison data to lenders.
- **Audit Trail:** An append-only auditLogs table provides non-repudiation for administrative, underwriting, and authorization events.

5. Verification Suite & Test Results

Test Suite / Command	Scope & Details	Result
npx tsc --project convex/tsconfig.json	Static type checking of Convex schema, auth helpers, health query, and generated APIs	**PASSED (0 errors)**
npm run build (tsc -b && vite build)	Full frontend production compilation (1,845 modules transformed)	**PASSED (8.18s)**
pytest tests/ -q	Python backend comparison, admin, and mock API regression suite	**6/6 PASSED**
Live Convex Cloud Connectivity	Network sync and deployment provisioning (`npx convex dev`)	**PENDING (Requires Cloud Credentials)**

6. Next Steps & Implementation Roadmap

Step 1: Obtain leadership sign-off on tenancy scope (single vs. multi-agency) and identity provider (Clerk/Auth0).

Step 2: Run npx convex dev to link the live Convex Cloud deployment and generate live deployment credentials.

Step 3: Model the single-loan transaction slice: Borrower Origination → Agency Rate/Margin → Lender Review → Approval in integer pesewas.